```
iex> self()
#PID<0.109.0>

iex> pid = spawn(fn -> IO.puts("Hello from #{inspect self()}") end)
Hello from #PID<0.112.0>
#PID<0.112.0>

iex> Process.alive?(pid)
false
```

We pass a function into spawn/1 that prints out some information about the executing process. Your exact numbers will be different than mine, but notice that the spawned function prints out as 0.112.0 and the originating process is 0.109.0. This is called a process ID (PID) and is one of the core data types in Elixir. The difference in PIDs proves that the spawned function is actually executing inside of a different process.

This spawned process would be useful if we wanted to fire off some asynchronous code, but it's not really that useful right now. We need to be able to send messages into the process and receive responses from it in order to turn it into a useful tool. The receive function lets us do just that. And, to send a message to the process, we'll use the send function.

Type this code into your IEx session:

```
iex> pid = spawn(fn ->
  receive do
    :hello -> IO.puts("Hello World")
    {:hello, name} -> IO.puts("Hello #{name}")
  end
end)

iex> Process.alive?(pid)
true

iex> send(pid, :hello)
Hello World
:hello

iex> Process.alive?(pid)
false
```

We were able to process the message :hello and see that the correct output was printed. If you try the example again with send(pid, {:hello, "Your Name"}), you'll see that it responds with a different message. The receive function uses pattern matching to determine which code to run, just like you're already familiar with from Chapter 4, Pattern Matching Your Way to Success, on page 49.

We aren't going to do the exercise here, but if you wanted to receive a response from the spawned server, you would use send to respond back to the originating